JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class LogRecord

java.lang.Object
    java.util.logging.LogRecord

All Implemented Interfaces:

Serializable

```
public class LogRecord
extends Object
implements Serializable
```

LogRecord objects are used to pass logging requests between the logging framework and individual log Handlers.

When a LogRecord is passed into the logging framework it logically belongs to the framework and should no longer be used or updated by the client application.

Note that if the client application has not specified an explicit source method name and source class name, then the LogRecord class will infer them automatically when they are first accessed (due to a call on getSourceMethodName or getSourceClassName) by analyzing the call stack. Therefore, if a logging Handler wants to pass off a LogRecord to another thread, or to transmit it over RMI, and if it wishes to subsequently obtain method name or class name information it should call one of getSourceClassName or getSourceMethodName to force the values to be filled in.

Serialization notes:

- The LogRecord class is serializable.
- Because objects in the parameters array may not be serializable, during serialization all objects in the parameters array are written as the corresponding Strings (using Object.toString).
- The ResourceBundle is not transmitted as part of the serialized form, but the resource bundle name is, and the recipient object's readObject method will attempt to locate a suitable resource bundle.

Since:

1.4

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
LogRecord(Level level, String msg)	Construct a LogRecord with the given level and message values.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
Instant	**getInstant**()	Gets the instant that the event occurred.
Level	**getLevel**()	Get the logging message level, for example Level.SEVERE.
String	**getLoggerName**()	Get the source Logger's name.
long	**getLongThreadID**()	Get a thread identifier for the thread where message originated
String	**getMessage**()	Get the "raw" log message, before localization or formatting.
long	**getMillis**()	Get truncated event time in milliseconds since 1970.
Object[]	**getParameters**()	Get the parameters to the log message.
ResourceBundle	**getResourceBundle**()	Get the localization resource bundle
String	**getResourceBundleName**()	Get the localization resource bundle name
long	**getSequenceNumber**()	Get the sequence number.
String	**getSourceClassName**()	

		Get the name of the class that (allegedly) issued the logging request.
String	**getSourceMethodName**()	Get the name of the method that (allegedly) issued the logging request.
int	**getThreadID**()	**Deprecated.** Values returned by this method may be synthesized, and may not correspond to the actual thread id, use getLongThreadID() instead.
Throwable	**getThrown**()	Get any throwable associated with the log record.
void	**setInstant**(Instant instant)	Sets the instant that the event occurred.
void	**setLevel**(Level level)	Set the logging message level, for example Level.SEVERE.
void	**setLoggerName**(String name)	Set the source Logger's name.
LogRecord	**setLongThreadID**(long longThreadID)	Set an identifier for the thread where the message originated.
void	**setMessage**(String message)	Set the "raw" log message, before localization or formatting.
void	**setMillis**(long millis)	**Deprecated.** LogRecord maintains timestamps with

		nanosecond resolution, using `Instant` values.
void	**setParameters**(Object[] parameters)	Set the parameters to the log message.
void	**setResourceBundle**(ResourceBundle bundle)	Set the localization resource bundle.
void	**setResourceBundleName**(String name)	Set the localization resource bundle name.
void	**setSequenceNumber**(long seq)	Set the sequence number.
void	**setSourceClassName**(String sourceClassName)	Set the name of the class that (allegedly) issued the logging request.
void	**setSourceMethodName**(String sourceMethodName	Set the name of the method that (allegedly) issued the logging request.
void	**setThreadID**(int threadID)	**Deprecated.** This method doesn't allow to pass a long thread id, use `setLongThreadID(long)` instead.
void	**setThrown**(Throwable thrown)	Set a throwable associated with the log event.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Constructor Details

LogRecord

```
public LogRecord(Level level,
                 String msg)
```

Construct a LogRecord with the given level and message values.

The sequence property will be initialized with a new unique value. These sequence values are allocated in increasing order within a VM.

Since JDK 9, the event time is represented by an `Instant`. The instant property will be initialized to the current instant, using the best available clock on the system.

The thread ID property will be initialized with a unique ID for the current thread.

All other properties will be initialized to "null".

Parameters:

`level` - a logging level value

`msg` - the raw non-localized logging message (may be null)

See Also:

`Clock.systemUTC()`

Method Details

getLoggerName

```
public String getLoggerName()
```

Get the source Logger's name.

Returns:

source logger name (may be null)

setLoggerName

```
public void setLoggerName(String name)
```

Set the source Logger's name.

Parameters:

`name` - the source logger name (may be null)

getResourceBundle

```
public ResourceBundle getResourceBundle()
```

Get the localization resource bundle

This is the ResourceBundle that should be used to localize the message string before formatting it. The result may be null if the message is not localizable, or if no suitable ResourceBundle is available.

Returns:

the localization resource bundle

setResourceBundle

```
public void setResourceBundle(ResourceBundle bundle)
```

Set the localization resource bundle.

Parameters:

`bundle` - localization bundle (may be null)

getResourceBundleName

```
public String getResourceBundleName()
```

Get the localization resource bundle name

This is the name for the ResourceBundle that should be used to localize the message string before formatting it. The result may be null if the message is not localizable.

Returns:

the localization resource bundle name

setResourceBundleName

```
public void setResourceBundleName(String name)
```

Set the localization resource bundle name.

Parameters:

`name` - localization bundle name (may be null)

getLevel

```
public Level getLevel()
```

Get the logging message level, for example Level.SEVERE.

Returns:

the logging message level

setLevel

```
public void setLevel(Level level)
```

Set the logging message level, for example Level.SEVERE.

Parameters:

`level` - the logging message level

getSequenceNumber

```
public long getSequenceNumber()
```

Get the sequence number.

Sequence numbers are normally assigned in the LogRecord constructor, which assigns unique sequence numbers to each new LogRecord in increasing order.

Returns:

the sequence number

setSequenceNumber

```
public void setSequenceNumber(long seq)
```

Set the sequence number.

Sequence numbers are normally assigned in the LogRecord constructor, so it should not normally be necessary to use this method.

Parameters:

`seq` - the sequence number

getSourceClassName

```
public String getSourceClassName()
```

Get the name of the class that (allegedly) issued the logging request.

Note that this sourceClassName is not verified and may be spoofed. This information may either have been provided as part of the logging call, or it may have been inferred automatically by the logging framework. In the latter case, the information may only be approximate and may in fact describe an earlier call on the stack frame.

May be null if no information could be obtained.

Returns:

the source class name

setSourceClassName

```
public void setSourceClassName(String sourceClassName)
```

Set the name of the class that (allegedly) issued the logging request.

Parameters:

`sourceClassName` - the source class name (may be null)

getSourceMethodName

```
public String getSourceMethodName()
```

Get the name of the method that (allegedly) issued the logging request.

Note that this sourceMethodName is not verified and may be spoofed. This information may either have been provided as part of the logging call, or it may have been inferred automatically by the logging framework. In the latter case, the information may only be approximate and may in fact describe an earlier call on the stack frame.

May be null if no information could be obtained.

Returns:

the source method name

setSourceMethodName

```
public void setSourceMethodName(String sourceMethodName)
```

Set the name of the method that (allegedly) issued the logging request.

Parameters:

`sourceMethodName` - the source method name (may be null)

getMessage

```
public String getMessage()
```

Get the "raw" log message, before localization or formatting.

May be null, which is equivalent to the empty string "".

This message may be either the final text or a localization key.

During formatting, if the source logger has a localization ResourceBundle and if that ResourceBundle has an entry for this message string, then the message string is replaced with the localized value.

Returns:

the raw message string

setMessage

```
public void setMessage(String message)
```

Set the "raw" log message, before localization or formatting.

Parameters:

`message` - the raw message string (may be null)

getParameters

```
public Object[] getParameters()
```

Get the parameters to the log message.

Returns:

the log message parameters. May be null if there are no parameters.

setParameters

```
public void setParameters(Object[] parameters)
```

Set the parameters to the log message.

Parameters:

`parameters` - the log message parameters. (may be null)

getThreadID

```
@Deprecated(since="16")
public int getThreadID()
```

Deprecated.
Values returned by this method may be synthesized, and may not correspond to the actual thread id, use `getLongThreadID()` instead.

Get an identifier for the thread where the message originated.

This is a thread identifier within the Java VM and may or may not map to any operating system ID.

Returns:

thread ID

setThreadID

```
@Deprecated(since="16")
public void setThreadID(int threadID)
```

Deprecated.
This method doesn't allow to pass a long thread id, use `setLongThreadID(long)` instead.

Set an identifier for the thread where the message originated.

Parameters:

`threadID` - the thread ID

getLongThreadID

```
public long getLongThreadID()
```

Get a thread identifier for the thread where message originated

This is a thread identifier within the Java VM and may or may not map to any operating system ID.

Returns:

thread ID

Since:

16

setLongThreadID

```
public LogRecord setLongThreadID(long longThreadID)
```

Set an identifier for the thread where the message originated.

Parameters:

`longThreadID` - the thread ID

Returns:

this LogRecord

Since:

16

getMillis

```
public long getMillis()
```

Get truncated event time in milliseconds since 1970.

API Note:

To get the full nanosecond resolution event time, use `getInstant()`.

Implementation Requirements:

This is equivalent to calling `getInstant().toEpochMilli()`.

Returns:

truncated event time in millis since 1970

See Also:

`getInstant()`

setMillis

```
@Deprecated
public void setMillis(long millis)
```

Deprecated.
LogRecord maintains timestamps with nanosecond resolution, using `Instant` values. For this reason, `setInstant()` should be used in preference to `setMillis()`.

Set event time.

Implementation Requirements:

This is equivalent to calling `setInstant(Instant.ofEpochMilli(millis))`.

Parameters:

`millis` - event time in millis since 1970.

See Also:

`setInstant(java.time.Instant)`

getInstant

```
public Instant getInstant()
```

Gets the instant that the event occurred.

Returns:

the instant that the event occurred.

Since:

9

setInstant

```
public void setInstant(Instant instant)
```

Sets the instant that the event occurred.

If the given `instant` represents a point on the time-line too far in the future or past to fit in a `long` milliseconds and nanoseconds adjustment, then an `ArithmeticException` will be thrown.

Parameters:

`instant` - the instant that the event occurred.

Throws:

`NullPointerException` - if `instant` is null.

`ArithmeticException` - if numeric overflow would occur while calling `instant.toEpochMilli()`.

Since:

9

getThrown

```
public Throwable getThrown()
```

Get any throwable associated with the log record.

If the event involved an exception, this will be the exception object. Otherwise null.

Returns:

a throwable

setThrown

```
public void setThrown(Throwable thrown)
```

Set a throwable associated with the log event.

Parameters:

`thrown` - a throwable (may be null)

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA

94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.